

Artificial Neural Network NSGA-II (ANN-NSGA-II) for the Multi-Objective Assignment Problem

Christoffer Jensen

25134539

07/01/2025

1. INTRODUCTION

The assignment problem is one of the first studied combinatorial optimization problems and was first introduced by Gaspard Monge in 1784 (Schrijver, 2005). The goal of the problem is to minimize the cost of assigning n tasks to n agents where each agent can take exactly one task, and each combination of task and agent have an associated cost. If we define x_{ij} as a binary variable taking the value 1 if task i is assigned to agent j and 0 otherwise and define the cost matrix $C = \{c_{ij}\}$ where c_{ij} is the associated cost of assigning task i to agent j , for $i, j = 1, 2, \dots, n$, then the assignment problem can be defined as

$$\begin{aligned} \min \quad & \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{j=1}^n x_{ij} = 1 \\ & \sum_{i=1}^n x_{ij} = 1 \end{aligned}$$

(Schrijver, 2005). The assignment problem describes a general setting which occurs in a multitude of fields. Any effective methods developed for solving the assignment problem would therefore have a vast area of application. This consequently led to an extensive development of algorithms made to tackle the problem (Schrijver, 2005). As methods were developed for the original version of the assignment problem, researchers started to focus on developing algorithms for alterations of the original problem, such as the multi-objective assignment problem (Cho, et al., 2017).

A multi-objective assignment problem tries to find an assignment which minimizes multiple objective functions opposed to the original problem which only tries to minimize one, i.e. cost. A multi-objective problem can in general be defined as

$$\min_{x \in X} (f_1(x), f_2(x), \dots, f_k(x)) \quad (1)$$

where X is the feasible set of solutions and k is the number of objectives. Given a multi-objective assignment problem, one of the objective functions would be to minimize the cost and X would partially be defined by the constraints of the assignment problem.

In real-world problems it is rare that we only have one criterion to optimize assignment. Take for example the setting where a construction company has n jobs which need to be carried out and the company has n workers they can assign one job each. Furthermore, the company knows how long each worker needs to finish each job. If the company simply wanted an assignment which minimizes the sum of total working hours, then this would be a case of the original assignment problem. Here, it would however be expected that the company also would focus on criteria such as minimizing work imbalance, minimizing traveling time, etc. The

problem then becomes a multi-objective assignment problem. Finding an optimal solution to such a problem proposes a new challenge, since the objectives might be conflicting, i.e. minimizing one would increase the other. This means that there is no one clear optimal solution. To solve a multi-objective optimization (MOO) problem, you would therefore first need to define a method for comparing the quality of the solutions. One such method is the pareto front method. Given two solutions $x, x^* \in X$ from (1) it says x pareto dominates x^* if

$$f_i(x) \leq f_i(x^*) \quad \forall i \in \{1, 2, \dots, m\}$$

and

$$\exists i \in \{1, 2, \dots, m\} : f_i(x) < f_i(x^*).$$

Furthermore, if we define $Y := f(X)$ then the subset of all non-dominated objective vectors in Y is called the Pareto front (Emmerich & Deutz, 2018). Solutions with objective functions in the Pareto front can then be interpreted as optimal compared to non-Pareto front solutions. Solving a MOO problem using this method would then consist of trying to estimate the corresponding Pareto front. This method allows for a clear comparison of the trade-offs between the different Pareto optimal solutions, ensuring high interpretability in decision making. The downside of this method is that there is no clear way of choosing between the Pareto optimal solutions (Emmerich & Deutz, 2018).

Since Pareto fronts consist of a set of non-dominated solutions, evolutionary algorithms are particularly well suited for estimating them, as their population-based search allows for multiple trade-offs to be explored simultaneously, producing a diverse approximation of the front in a single run. As a result, a wide range of pareto-based multi-objective evolutionary algorithms (MOEAs) have been developed to address MOO problems, with the nondominated sorting genetic algorithm II (NSGA-II) being one of the most prominent (Hanne & Moghaddam, 2025).

In this report, I will introduce and evaluate an Artificial Neural Network NSGA-II (ANN-NSGA-II) algorithm developed for the multi-objective assignment problem. The algorithm is an Artificial Neural Network (ANN) mutation modification of the well-established NSGA-II multi-objective evolutionary algorithm.

The structure of the report is as follows: section 2 describes previous research regarding evolutionary algorithms for MOO with a special focus on the NSGA-II algorithm. Section 3 describes the ANN-NSGA-II algorithm. Section 4 describes the efforts made to evaluate the model. Section 5 and 6 respectively describes and discusses the results obtained from the evaluation, and finally Section 6 concludes the project.

2. BACKGROUND

As discussed in the previous section, the population-based structure of evolutionary algorithms makes them well suited for solving MOO problems through Pareto front estimation. However, this is

not the only method by which evolutionary algorithms can address MOO. Much research has explored a variety of MOEAs that use different methods beyond Pareto dominance, including decomposition, indicator-based, and hybrid methods (Hanne & Moghaddam, 2025). An example of a significant non-Pareto based MOEA is the MOEA/D which uses a decomposition-based method. A decomposition-based method is particularly well-suited for a many-objective optimization problem. However, Pareto-dominance-based MOEAs historically constitute the largest class of MOEAs with the NSGA-II being the most prominent of these (Hanne & Moghaddam, 2025).

The NSGA-II is an improvement of the previous NSGA which: suffered from high computational cost due to nested loops in its sorting procedure, relied on a parameter-based mechanism to ensure population diversity, and lacked elitism. The NSGA-II improved the computational cost by introducing a fast non-dominated sorting algorithm and introduced crowding distance, a parameter-free mechanism to maintain a high population diversity. It further combined these to define the crowded comparison-operator which insures a level of elitism (Hanne & Moghaddam, 2025). Since it was first proposed, the NSGA-II has been applied to a multitude of MOO problems including multi-objective combinatorial optimization problems (MOCOPs) and modified extensively (Verma, et al., 2021). In some cases, researchers have modified NSGA-II by changing either selection, mutation, crossover, crowding distance, etc. and in other cases hybrid versions have been created by combining NSGA-II with other algorithms such as local search or including greedy or constructive heuristics. When implemented on MOCOP such as the multi-objective assignment problem, the crossover and mutation operators of NSGA-II is often modified to ensure permutation-based encoding for the solutions (Verma, et al., 2021).

Recent studies have investigated the incorporation of deep learning, a prominent branch of computational intelligence, into evolutionary algorithms (EAs). Deep learning methods are particularly well suited for extracting meaningful patterns from complex data. By learning from data generated during the evolutionary process, deep learning models can extract informative patterns that may be useful for guiding and improving the optimization process of EAs (Bian, et al., 2026). Motivated by this emerging trend of integrating deep learning into EAs, this study introduces ANN-NSGA-II, an extension of the NSGA-II algorithm incorporating an ANN-based mutation operator. This mutation operator combines a conventional random binary-swap mutation with an ANN-based binary-swap mutation, where the ANN is trained during the initial iterations of the algorithm using information gathered from successful random swap operations. Since the ANN-NSGA-II is developed for the multi-objective assignment problem, it will also include modifications to the crossover and mutation which ensures a permutation-based encoding of the individuals.

3. METHOD

As mentioned, the ANN-NSGA-II is an ANN mutation modification of the NSGA-II. The NSGA-II is a pareto-based MOEA which uses a fast non-dominated sorting algorithm and crowding distance to efficiently generate a diverse approximation of the pareto front. To clarify the structure of the proposed ANN-NSGA-II algorithm, I first describe the standard NSGA-II framework, followed by a detailed explanation of the modifications introduced by ANN-NSGA-II.

3.1 NSGA-II

Since the NSGA-II is a MOEA, it evolves a population of individuals (i.e. solutions) through multiple generations (i.e.

iterations) consisting of the following steps: (step 1) select individuals from the current population for reproduction, (step 2) generate offspring through a crossover between selected individuals, (step 3) mutate the offspring, (step 4) select individuals for survival into the next generation (Deb, et al., 2002).

The main improvements introduced in NSGA-II relative to its predecessor (i.e. NSGA) concern the selection of individuals for reproduction (step 1) and the selection of individuals for survival into the next generation (step 4). These are achieved through the introduction of a fast non-dominated sorting procedure, the crowding distance measure to preserve population diversity and the crowded-comparison operator. In the original NSGA-II paper, the authors do not prescribe specific crossover (step 2) or mutation (step 3) operators but instead rely on problem-specific operators (Deb, et al., 2002).

3.1.1 Fast non-dominated sorting

Both NSGA-II and its predecessor (i.e. NSGA) select the individuals for survival (step 4) based on a pareto non-dominance sorting system. However, the NSGA-II is using an improved sorting algorithm compared to the NSGA. Here, individuals are sorted according to their level of non-dominance, defined by the number of non-dominated fronts that must be removed before a given individual becomes non-dominated. In each generation, NSGA-II applies this ranking to the combined set of parent and offspring individuals, denoted by R . The ranking procedure begins by iterating over all the individuals of R . For each individual $p \in R$ two entities are calculated: (1) domination count n_p , the number of individuals which dominates p , (2) domination set S_p , the set of individuals dominated by p . The individuals with domination count equal to zero will be added to the first non-dominated front F_1 . Now, for each individual $p \in F_1$ the algorithm looks at the members $q \in S_p$ and reduces their domination count by one. Consequently, if any member q now have a domination count of zero (i.e. $n_q = 0$) it will be added to the second non-dominated front F_2 , since removing F_1 leaves it non-dominated. This process is continued for the individuals in F_2 and the third front, F_3 , is identified. This process continues until all fronts are identified. The overall complexity of this algorithm is $O(MN^2)$ where N is the total number of individuals and M is the number of objectives. This is a significantly improved complexity compared to the complexity of the sorting algorithm in NSGA which is $O(MN^3)$ (Deb, et al., 2002).

3.1.2 Crowding distance

After sorting the individuals into fronts, the NSGA-II then iterates over the fronts in ascending order. When at front F_i let P be the individuals already selected for survival. If the size of $F_i + P$ does not exceed the population size, then all individuals from F_i is added to P . If the size of P after adding F_i does not equal the population size, the algorithm continues to F_{i+1} . However, if the size of $F_i + P$ does exceed the population size, individuals from F_i will be added to P based on crowding distance in a descending order until P reaches the population size. For an individual $p \in F_i$ the crowding distance is a measure of how closely clustered p is with its neighbours in F_i . The measure is given by the sum of the absolute normalized difference in each objective between the two individuals adjacent to p in F_i . The boundary points in each objective are given an infinitely high crowding distance. Since the individuals are selected based on crowding distance in a descending order, this helps maintain a high population diversity (Deb, et al., 2002).

3.1.3 Crowded comparison operator

The non-dominated sorting and crowding distance are also used for selecting individuals for reproduction (step 1). The NSGA-II uses a binary-tournament selection, but instead of basing the tournament on fitness it defines a crowded-comparison operator $\prec_n$ to compare the individuals. Given two individuals p and q , the crowded-comparison operator assumes that each individual has an assigned front (i.e. $p \in F_i$ and $q \in F_j$) and a crowding distance. If this is the case, then $p \prec_n q$ if

$$i \leq j \vee (i = j \wedge p_{cd} > q_{cd})$$

where p_{cd} and q_{cd} is the crowding distance for p and q respectively. This crowded-comparison operator is defined to further ensure diversity in the population. In the NSGA, this selection was based on the concept of sharing. Sharing was also developed to ensure diversity in the population but required a sharing parameter σ_{share} . Since the diversity mechanisms of the NSGA-II are parameter-free they are preferred compared to the NSGA (Deb, et al., 2002).

3.2 ANN-NSGA-II

The ANN-NSGA-II is also a MOEA and therefore evolves a population through the same 4 steps as the NSGA-II. The structure of ANN-NSGA-II follows that of NSGA-II for steps 1 and 4. However, since the ANN-NSGA-II is developed for a MOCOP (i.e. the multi-objective assignment problem) it requires problem specific encoding of individuals, crossover operator (step 2) and mutations operator (step 3).

3.2.1 Encoding and initialization

To accommodate the constraints of the multi-objective assignment problem, the ANN-NSGA-II uses a permutation-based encoding for the individuals. That is, each individual is represented as an integer vector (i.e. chromosome) with length equal to the number of jobs. Then each entry of the chromosome (i.e. gene) represents a job, and the value of the gene represents the agent assigned to this job. In this manner each individual's chromosome gives a permutation of the possible agents. For a setting where the total number of jobs and agents are $n = 4$, the following valid individual $p = [3, 4, 1, 2]$ would indicate that job 1 is assigned to agent 3, job 2 to agent 4, job 3 to agent 1 and job 4 to agent 2.

In the first generation of the ANN-NSGA-II, the chromosomes of the individuals are initialized through n random uniform draws without replacement from the sample of all agents, where n is the total number of agents.

3.2.2 Crossover

The most important feature of the crossover in the ANN-NSGA-II is that it preserves the permutation-based encoding. There exist multiple different crossovers which obtain this goal, such as the partially mapped crossover (PMX), ordered crossover (OX), cycle crossover (CX), etc. PMX is recognized as one of the most popular and effective of these crossovers and is therefore chosen as the crossover in ANN-NSGA-II (Ting, et al., 2010). The PMX was first described in (Goldberg & Lingle, 1985) and works in the following way: given two parents with permutation-based encoding as described above, two cut points are selected uniformly at random along the chromosome. An offspring is then constructed by copying the subsequence between the cut points (*middle*) from one parent (the *mapping parent*) directly into the corresponding positions of the offspring. The remaining positions (*outer*) are filled iteratively using genes from the other parent (the *filling parent*). If a duplicate occurs when generating the *outer* section, the value of the gene which is a duplicate is used to index the chromosome of the

mapping parent. The obtained gene from the *mapping parent* is then used to index the *filling parent*. If this new gene is also a duplicate, then the mapping process is repeated until a non-duplicate is found.

3.2.3 Mutation

As mentioned, the mutation operator of the ANN-NSGA-II is defined as a combination of a random binary-swap and an ANN binary-swap mutation. Initially, only the random binary-swap mutation will be responsible for performing the swaps. This is also called the warmup phase. After the warmup phase, the mutation operator enters the ramp up phase. This phase is defined through a number of generations where the ANN binary-swap mutation is used with an increasing probability. After the ramp up phase, the ANN binary-swap mutation is used for the remaining generations with the probability reached at the end of the ramp up phase. In each generation it is recorded whether the individuals which survive to the next generation are parents or offspring. If they are offspring, the chromosome of the offspring before the swap mutation and the actual swap performed on the offspring is added to a training dataset used to train the ANN binary-swap mutation. The training dataset has a maximum size n_{max} , meaning the ANN binary-swap mutation is only trained on the n_{max} most recent successful swaps. The exact number of generations for the warmup and ramp up phase, the maximum probability of using the ANN binary-swap and the size of n_{max} are left as hyperparameters.

3.2.3.1 Random binary-swap

The random binary-swap mutation simply selects two genes in the chromosome of the individual and swaps their values. This guarantees that permutation-based encoding is preserved after the mutation.

3.2.3.2 ANN binary-swap

The ANN binary-swap mutation consists of a multilayer perceptron (MLP) with a multi-head output. The MLP has two hidden layers each with 128 neurons and ReLU activation. The input of the MLP is a one-hot encoded vector of the permutation-based chromosomes. Given a chromosome of length n (i.e. assignment problem with size n), the one-hot encoding will be of length n^2 with the first n entries indicating which agent will be assigned to the first job, the second n entries indicating which agent will be assigned to the second job etc. The number of neurons in the input layer of the MLP is therefore determined based on the size of the assignment problem. The MLP has two output heads of length n (i.e. size of assignment problem) each with a Softmax activation with temperature scaling. Applying a Softmax function allows the outputs of the two heads to be interpreted as probability distributions over the n chromosome indices. The obtained distributions are then used to parameterize multinomial sampling for selecting the two swap indices. To ensure a valid swap operation, the distribution corresponding to the second index is masked based on the outcome of the first draw, thereby preventing the selection of the same index twice. The MLP is trained using the Adam optimizer with a cross entropy-loss. The MLP is trained at the end of each generation using the collected samples of chromosomes and swaps.

3.2.4 Archive

Many MOEAs maintain an external archive to store and update an approximation of the Pareto front throughout the evolutionary process. However, this is not required for ANN-NSGA-II, since it uses the same fast non-dominated sorting and elitist selection as the NSGA-II. This mechanism ensures that individuals are selected based on Pareto non-dominance and diversity, such that the

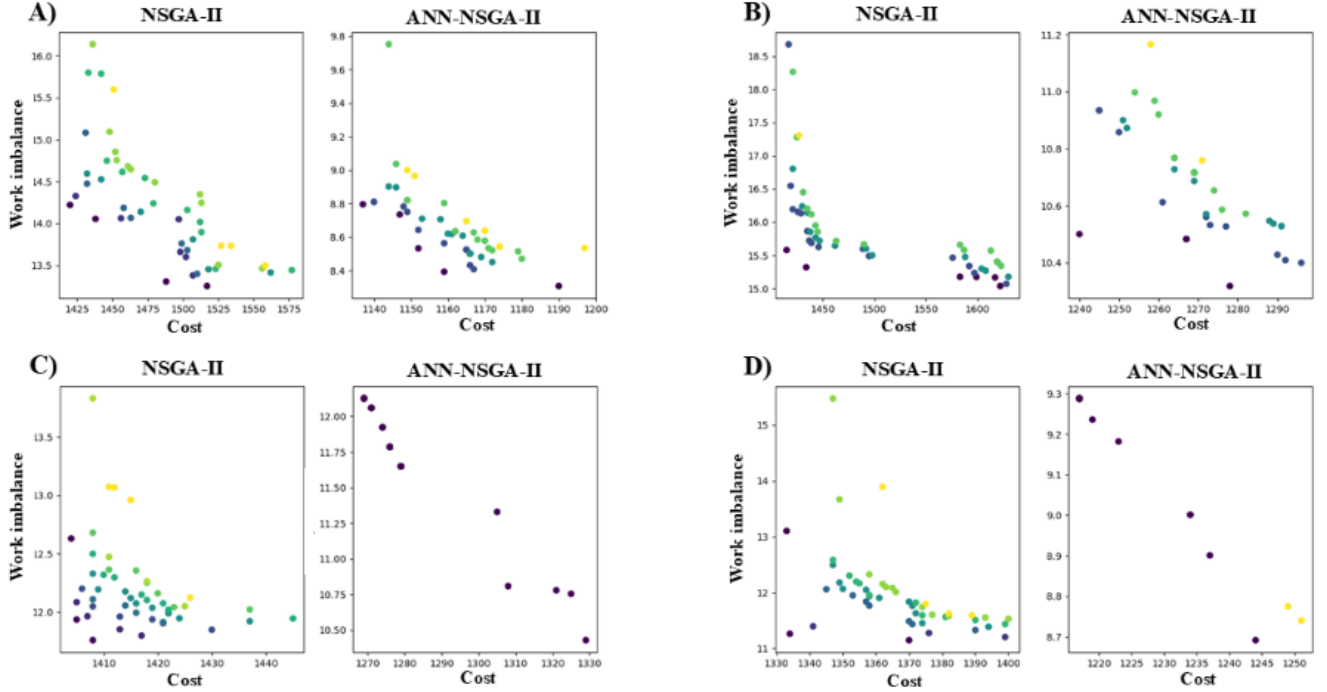


Figure 1: The final population for NSGA-II and ANN-NSGA-II for four different runs on the assignment problem with dimension 200. The fronts in the populations are indicated by color, with dark purple indicating F_1 , i.e. the pareto front estimation.

population itself provides an approximation of the Pareto front. Consequently, the Pareto front estimated by ANN-NSGA-II is given by the set of non-dominated individuals in the final population.

4. EXPERIMENTAL SETUP

The performance of the ANN-NSGA-II algorithm is evaluated on two instances of the assignment problem obtained from the Operation Research (OR) library (Beasley, 1990). The two instances are the assignment problems of dimension 100 and 200. The results are compared against those produced by the original NSGA-II algorithm with PMX crossover and random binary-swap mutation. The original NSGA-II is also initialized as the ANN-NSGA-II. Each instance of the assignment problem from the OR library consists of a cost matrix. Since ANN-NSGA-II is developed for a multi-objective assignment problem, we want to define at least two objectives based on the cost matrix with one being the minimization of assignment cost, as defined in the introduction. Here, I define a second objective of minimizing work imbalance, given by the variance of the cost assigned to each agent. If we have an assignment problem with a cost matrix $C = \{c_{ij}\}$ of dimension n and we again define x_{ij} as a binary variable taking value 1 if task i is assigned to agent j and 0 otherwise, for $i, j = 1, 2, \dots, n$, then this second objective can be defined as

$$\min \sum_{i=1}^n (W_i - \bar{W})^2$$

where

$$W_i = \sum_{j=1}^n c_{ij} x_{ij}, \quad \bar{W} = \frac{1}{n} \sum_{i=1}^n W_i.$$

Before initiating the evaluation, we need to define the hyperparameters for both algorithms. NSGA-II has only one hyperparameter, i.e. the population size. For both the NSGA-II and

ANN-NSGA-II the population size is set to 50. For ANN-NSGA-II we further set the warmup phase as 20% of the total generations, the ramp up phase as 60% of the total generations, the maximum probability of ANN binary-swap mutation is set to 70%, the n_{max} for the training dataset is set to 100, the Softmax temperature is set to 2, the batch size for training the ANN binary-swap mutation is set to 32, for each training one epoch is used and the learning rate of the Adam optimizer is set to 0.001.

As both algorithms rely on random initialization, 10 independent runs were performed for each algorithm and problem instance. For each run both algorithms evolve their population over 3000 generations. The results were then compared on a per-run basis to reduce the influence of stochastic effects arising from initialization. For each run the hypervolume of the pareto front estimation for both methods was calculated using a reference point slightly worse than their combined nadir point.

5. RESULTS

In 7 out of the 10 runs done on the assignment problem with dimension 100, ANN-NSGA-II scored a higher hypervolume than NSGA-II. Similarly, in 8 out of the 10 runs on the assignment problem with dimension 200, ANN-NSGA-II scored a higher hypervolume than NSGA-II. This indicates that ANN-NSGA-II gives a better estimation of the pareto front of the defined multi-objective assignment problem than NSGA-II. However, a visual examination of the final populations for both methods shows that even though ANN-NSGA-II scores a higher hypervolume it frequently struggles with maintaining a high diversity in its population compared to NSGA-II. This is illustrated in figure 1, which shows the final population for both methods for four runs on the assignment problem with dimension 200. In each run the ANN-NSGA-II scored a higher hypervolume than NSGA-II. However, in run C) and D) ANN-NSGA-II clearly struggles with maintaining a diverse population with its entire population (i.e. 50 individuals) collapsing into 10 and 8 different individuals respectively.

6. DISCUSSION

The above results reflect that the added exploitation introduced by the ANN binary-swap mutation in ANN-NSGA-II has made ANN-NSGA-II better suited for estimating the pareto front of the multi-objective assignment problem, as defined in the experimental setup, than NSGA-II. Claiming that this makes ANN-NSGA-II a superior algorithm compared to NSGA-II would, however, be a very bold claim. The performance of the algorithms is evaluated using only a single performance metric, namely the hypervolume. Often, when evaluating the performance of novel pareto-based MOEAs, problems with a known pareto front are used, which enables the use of a broader range of performance metrics, as the availability of the true front allows direct assessment of convergence (Nuh, et al., 2021). However, the true pareto fronts for the problems used in this project are unknown, limiting the number of applicable performance metrics. Nonetheless, the hypervolume indicator is one of the most widely used performance metrics for MOEAs, which means that the obtained results still hold significance (Guerreiro, et al., 2021). Furthermore, the algorithms are only evaluated on two instances of the multi-objective assignment problem both with a relatively low dimensionality. The main reason for this is directly connected to a limiting factor of the ANN-NSGA-II algorithm - its complexity. The introduction of the ANN binary-swap mutation significantly increases the complexity of the algorithm compared to NSGA-II. As a consequence, ANN-NSGA-II does not scale as well as NSGA-II.

Another potential limiting factor of ANN-NSGA-II is its ability to maintain population diversity. The added exploitation through the ANN binary-swap mutation increases the risk of premature convergence. As seen in the results, its population frequently collapses into a few individuals. However, this did not pose a significant issue within the experimental setup considered in this report, since ANN-NSGA-II still managed to produce pareto front estimations with a higher hypervolume than NSGA-II. However, it is likely that neither algorithm had fully converged after 3000 generations, and additional evolution could further improve their Pareto front estimations. For the ANN-NSGA-II a further evolution also increases the risk of population collapse, potentially leading to premature convergence from which further improvements are unlikely. This issue can be mitigated by limiting the exploitative influence of the ANN binary-swap mutation through appropriate hyperparameter tuning. However, overly restricting the ANN binary-swap mutation would reduce the distinction between ANN-NSGA-II and the standard NSGA-II. This clearly shows the importance of finding a trade-off between exploitation and exploration when including an adaptive mutation operator. It also exposes another limiting factor of ANN-NSGA-II, namely its dependence on a large number of hyperparameters. These strongly influence the performance of the algorithm, making thorough hyperparameter tuning essential for achieving optimal results. However, due to the high complexity of the algorithm, such tuning entails a substantial computational cost. This consideration is the reason why no extensive hyperparameter tuning was performed in the experimental setup of this project.

A final observation regarding both ANN-NSGA-II and NSGA-II in the present setup is that the effect of a binary-swap mutation diminishes as the dimensionality of the problem increases. If the algorithms were to be applied to higher-dimensional problems, it could therefore be beneficial to modify the mutation operator to affect more than two genes. This could, for example, be achieved by swapping multiple pairs of genes, swapping larger chromosome sequences, etc.

7. CONCLUSION

The introduced ANN-NSGA-II demonstrated improved performance relative to NSGA-II, producing Pareto front estimations with a higher hypervolume in 7 out of 10 and 8 out of 10 runs across the two problem instances. However, the use of only a single performance metric and a few low-dimensional problem instances limits the significance of the results. Furthermore, ANN-NSGA-II has a significantly higher complexity than NSGA-II and is highly dependent on hyperparameter tuning, both of which are generally undesirable characteristics. Nonetheless, the obtained results suggest that ANN-NSGA-II may represent an interesting direction for future research. In particular, it would be valuable to investigate how its performance is affected by a thorough hyperparameter tuning. Additionally, further studies could explore the scalability of ANN-NSGA-II when applied to larger and more complex problem instances.

8. CODE AVAILABILITY

The implementation of the ANN-NSGA-II and NSGA-II, as described in this project, is available through the following GitHub repository: <https://github.com/ChristofferJe/ANN-NSGA-II>. The overall structure of the code for the NSGA-II is inspired by the previous implementation of NSGA-II given in (Bao, et al., 2018).

9. BIBLIOGRAPHY

- Bao, P., Quan, T., Tho, Q., & Garg, A. (2018). NSGA-II Python. Retrieved from <https://github.com/baopng/NSGA-II>
- Beasley, J. (1990). OR-Library: Distributing Test Problems by Electronic Mail. *The Journal of the Operational Research Society*, 1069–1072. doi:<https://doi.org/10.2307/2582903>
- Bian, K., Zhang, J., Han, H., Zhou, J., Sun, Y., & Cheng, S. (2026). Deep-insights guided evolutionary algorithm for optimization. *Expert Systems with Applications*, 298, Part A. doi:<https://doi.org/10.1016/j.eswa.2025.129538>
- Cho, J.-H., Wang, Y., Chen, I.-R., Chan, K., & Swami, A. (2017). A Survey on Modeling and Optimizing Multi-Objective Systems. *IEEE Communications Surveys & Tutorials*, 19, 1867-1901. doi:10.1109/COMST.2017.2698366
- Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 182 - 197. doi:10.1109/4235.996017
- Emmerich, M., & Deutz, A. (2018). A tutorial on multiobjective optimization: fundamentals and evolutionary methods. *Natural Computing*, 585–609. doi:<https://doi.org/10.1007/s11047-018-9685-y>
- Goldberg, D., & Lingle, R. (1985). Alleles, loci, and the traveling salesman problem. *Proceedings of the 1st International Conference on Genetic Algorithms*, 154 - 159.
- Guerreiro, A., Fonseca, C., & Paquete, L. (2021). The Hypervolume Indicator: Problems and Algorithms. *ACM Computing Surveys*. doi:<https://doi.org/10.48550/arXiv.2005.00515>
- Hanne, T., & Moghaddam, M. (2025). A Review of the Evolution of Multi-Objective Evolutionary Algorithms. *Computers, Materials & Continua*, 4203-4236. doi:<https://doi.org/10.32604/cmc.2025.068087>

- Nuh, J., Koh, T., Baharom, S., Osman, M., & Kew, S. (2021). Performance Evaluation Metrics for Multi-Objective Evolutionary Algorithms in Search-Based Software Engineering: Systematic Literature Review. *Applied Sciences*. doi:<https://doi.org/10.3390/app11073117>
- Schrijver, A. (2005). On the History of Combinatorial Optimization (Till 1960). In G. Nemhauser , K. Aardal , & R. Weismantel , *Handbooks in Operations Research and Management Science: Discrete Optimization* (pp. 1-68). Elsevier.
- Tan, K., Lee, T., & Khor, E. (2002). Evolutionary Algorithms for Multi-Objective Optimization: Performance Assessments and Comparisons. *Artificial Intelligence Review*, 251–290. doi:<https://doi.org/10.1023/A:1015516501242>
- Ting, C.-K., Su, C.-H., & Lee, C.-N. (2010). Multi-parent extension of partially mapped crossover for combinatorial. *Expert Systems with Applications*, 1879–1886. doi:<https://doi.org/10.1016/j.eswa.2009.07.082>
- Verma, S., Pant, M., & Snasel, V. (2021). A Comprehensive Review on NSGA-II for Multi-Objective Combinatorial Optimization Problems. *IEEE Access*, 57757-57791. doi:10.1109/ACCESS.2021.3070634